

EasyFood - Missão 3

Persistência com PostgreSQL e Prisma

Na missão anterior, identificamos uma limitação importante da nossa arquitetura: os restaurantes cadastrados em memória desaparecem quando o servidor é reiniciado.

Agora chegou o momento de evoluir a EasyFood e substituir o armazenamento temporário por persistência em um banco de dados real.

```
GET /restaurants
POST /restaurants
PostgreSQL
Prisma
```

Parte 1 - PREPARE

Deixe sua EasyFood no mesmo ponto da aula

Antes de começar a missão, sua aplicação deve possuir as duas rotas abaixo:

```
GET /restaurants
POST /restaurants
```

Abra o terminal dentro do projeto e execute:

```
node server.js
```

Você deverá visualizar:

```
EasyFood rodando na porta 3000
```

Confirme que GET e POST estão funcionando antes de continuar.

Parte 2 - PREPARE O BANCO

1. Instale o PostgreSQL

Windows: baixe o instalador em <https://www.postgresql.org/download/windows/>. Mantenha as configurações padrão e defina uma senha para o usuário postgres.

macOS com Homebrew:

```
brew install postgresql@16
brew services start postgresql@16
```

Linux - Ubuntu/Debian:

```
sudo apt update
sudo apt install postgresql postgresql-contrib
sudo systemctl start postgresql
```

Valide a instalação:

```
psql --version
```

2. Crie o banco da EasyFood

No Windows, abra o SQL Shell (psql). No macOS/Linux, você pode executar:

```
psql -U postgres
```

Crie o banco:

```
CREATE DATABASE easyfood;
```

Depois, saia do psql:

```
\q
```

Parte 3 - CONECTE

3. Instale o Prisma

Dentro da pasta do projeto, execute:

```
npm install prisma --save-dev
npm install @prisma/client
```

Inicialize o Prisma:

```
npx prisma init
```

A estrutura do projeto passará a incluir:

```
easy-food/
|
+-- prisma/
|   +-- schema.prisma
|
+-- .env
+-- package.json
+-- package-lock.json
+-- server.js
```

4. Configure a conexão

Abra o arquivo .env e configure a conexão com o PostgreSQL:

```
DATABASE_URL="postgresql://postgres:SUA_SENHA_AQUI@localhost:5432/easyfood"
```

Substitua SUA_SENHA_AQUI pela senha definida na instalação.

No macOS, caso sua instalação local não utilize senha, a URL poderá ser:

```
DATABASE_URL="postgresql://postgres@localhost:5432/easyfood"
```

5. Defina o modelo Restaurant

Abra prisma/schema.prisma e configure:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Restaurant {
  id      Int      @id @default(autoincrement())
  name    String   @db.VarChar(150)
  category String? @db.VarChar(100)
  rating  Decimal? @db.Decimal(2, 1)
}
```

Agora o modelo que antes existia apenas como objetos JavaScript também possui uma representação persistente no banco.

6. Crie a tabela com uma migration

```
npx prisma migrate dev --name criar-tabela-restaurants
```

Esse comando lê o schema, gera o SQL necessário, aplica a alteração no banco e registra o histórico em prisma/migrations.

Parte 4 - POPULE O BANCO

7. Crie dados iniciais

Crie o arquivo prisma/seed.js:

```
const { PrismaClient } = require("@prisma/client");

const prisma = new PrismaClient();

async function main() {
  await prisma.restaurant.createMany({
    data: [
      { name: "Pizzaria Napoli", category: "Pizza", rating: 4.5 },
      { name: "Burger House", category: "Burger", rating: 4.2 },
      { name: "Sushi Express", category: "Japonesa", rating: 4.8 }
    ]
  });
}
```

```

    ]
  });

  console.log("Dados inseridos com sucesso!");
}

main()
  .catch((e) => {
    console.error(e);
    process.exit(1);
  })
  .finally(() => prisma.$disconnect());

```

Execute:

```
node prisma/seed.js
```

Parte 5 - EVOLUA A API

8. Conecte o server.js ao Prisma

Importe e inicialize o Prisma:

```

const { PrismaClient } = require("@prisma/client");

const prisma = new PrismaClient();

```

9. Altere o GET /restaurants

```

app.get("/restaurants", async (req, res) => {
  try {
    const restaurantes = await prisma.restaurant.findMany();
    res.json(restaurantes);
  } catch (error) {
    console.error("Erro ao buscar restaurantes:", error.message);
    res.status(500).json({ error: "Erro interno do servidor" });
  }
});

```

10. Altere o POST /restaurants

```

app.post("/restaurants", async (req, res) => {
  const { name, category, rating } = req.body;

  if (!name || !category) {
    return res.status(400).json({
      error: "Nome e categoria são obrigatórios"
    });
  }

  try {
    const novoRestaurante = await prisma.restaurant.create({
      data: { name, category, rating: rating || 0 }
    });

    res.status(201).json(novoRestaurante);
  } catch (error) {
    console.error("Erro ao cadastrar restaurante:", error.message);
    res.status(500).json({ error: "Erro interno do servidor" });
  }
});

```

Nossa arquitetura agora mudou de:

API -> ARRAY EM MEMÓRIA

para:

API -> PRISMA -> POSTGRESQL

Parte 6 - TESTE

11. Inicie o servidor

```
node server.js
```

12. Teste o GET

```
GET http://localhost:3000/restaurants
```

Os restaurantes inseridos no banco deverão aparecer.

13. Teste o POST

No Postman, faça:

```
POST http://localhost:3000/restaurants
```

No Body, selecione raw e JSON. Envie:

```
{
  "name": "Taco Loco",
  "category": "Mexicana",
  "rating": 4.3
}
```

Status esperado: 201 Created.

14. Faça o teste de persistência

Pare o servidor:

```
ctrl + c
```

Inicie novamente:

```
node server.js
```

Faça outra vez:

```
GET http://localhost:3000/restaurants
```

O Taco Loco continua aparecendo?

Se sim, a EasyFood agora possui persistência. Os dados não dependem mais da memória do processo Node.js.

Parte 7 - OBSERVE

15. Abra o Prisma Studio

```
npx prisma studio
```

O Prisma Studio abre uma interface visual para consultar e editar os registros do banco. Normalmente ele estará disponível em:

```
http://localhost:5555
```

Abra o model Restaurant e observe os dados cadastrados.

O que acabamos de fazer?

ANTES

```
Cliente
|
Express
|
Array em memória
|
Dados desaparecem ao reiniciar
```

AGORA

```
Cliente
|
Express
|
Prisma
|
PostgreSQL
|
Dados persistem
```

A API continua oferecendo GET /restaurants e POST /restaurants. O que mudou foi a implementação interna e a forma como os dados são armazenados.

Parte 8 - REGISTRE A NOVA DECISÃO

Na missão anterior, registramos que o armazenamento em memória era uma decisão temporária. Agora um dos critérios de revisão aconteceu: precisamos manter os dados entre reinicializações.

Crie:

```
docs/adr/ADR-002-persistencia-com-postgresql.md
```

Conteúdo esperado do ADR-002

Registre:

- ☐ Contexto
- ☐ Alternativas consideradas
- ☐ Decisão
- ☐ Justificativa
- ☐ Consequências positivas
- ☐ Consequências negativas / trade-offs
- ☐ Critérios de revisão

Não registre apenas que PostgreSQL foi escolhido. Explique por que essa decisão faz sentido para o contexto atual da EasyFood.

Parte 9 - PENSE COMO ARQUITETO

Responda:

- ☐ Qual problema arquitetural resolvemos ao adicionar persistência?
- ☐ Por que PostgreSQL faz sentido para a EasyFood neste momento?
- ☐ Qual é a responsabilidade do Prisma?
- ☐ O que acontece com a API se o banco ficar indisponível?
- ☐ Que vantagens ganhamos em relação ao array em memória?
- ☐ Que nova complexidade foi adicionada?
- ☐ Quais trade-offs surgiram com essa decisão?
- ☐ Nosso desenho arquitetural precisa ser atualizado?

Para concluir a Missão 3

- ☐ PostgreSQL instalado
- ☐ Banco easyfood criado
- ☐ Prisma instalado e inicializado
- ☐ DATABASE_URL configurada
- ☐ Model Restaurant criado
- ☐ Migration executada
- ☐ GET /restaurants consultando o PostgreSQL
- ☐ POST /restaurants salvando no PostgreSQL
- ☐ Teste de reinicialização realizado
- ☐ Prisma Studio testado
- ☐ ADR-002 criado

Próximo capítulo da EasyFood

Agora nossa aplicação possui persistência real:

Cliente -> API -> Prisma -> PostgreSQL

Mas nossa aplicação ainda concentra suas responsabilidades em um único projeto.

A próxima pergunta arquitetural será: como organizar e evoluir essa aplicação conforme novas responsabilidades e novos domínios surgirem?